

CS 223

Week 4 Lecture 1



Agenda

Announcements

Practice quiz review

Pass by pointer for non-array types

Struct pointers

NULL pointers

Return values

Standard I/O

Announcements

Lab this week: quiz!

- 1 hour
- You may bring a cheat sheet

HW#2 is due Thursday Sept 24 at midnight

Reminder:

- My office hours are Thursday 2:30pm-3:30pm (and by appointment) in Park 204
- Linh's are Fridays 3pm-5pm in Park 231

Quiz Review

Pointer operators: &, *

& - **address** operator, gives the address (its location in memory) of a variable

* - **dereference** operator, refers to the value of a pointer

```
#include <stdio.h>
```

```
int main() {  
    char c1 = 'a';  
    char* ptr;  
    ptr = &c1;  
    char c2 = *ptr;  
    printf("%c %c\n", c1, c2);  
    return 0;  
}
```

Pointer syntax

`<data_type>*`: declares a pointer type

`*<variable>`: Dereference a pointer to get the corresponding value at its address

`&<variable>`: address operator, gets the address where this variable is stored

Pointers

Be careful: * has two meanings!

```
void pass_by_pointer(int* v) {  
    *v = 10;  
}
```

Pass by pointer: draw stack

```
void pass_by_value(int v) {  
    v = 10;  
}  
  
void pass_by_pointer(int* v) {  
    *v = 10;  
}  
  
int main() {  
    int changeme = 0;  
    pass_by_value(changeme);  
    printf("The value is: %d\n", changeme);  
  
    pass_by_pointer(&changeme);  
    printf("The value is: %d\n", changeme);  
    return 0;  
}
```

Example: Pass by pointer

```
#include <stdio.h>

void inc(int* i) {
    int local = *i;
    local = local + 1;
    *i = local;
}

int main() {
    int i = 5;
    printf("Before: %d\n", i);
    inc(&i);
    printf("After: %d\n", i);
    return 0;
}
```

Struct pointers

Works like any other data type!

BUT use -> to access data inside the struct

Rule:

Use -> for struct **pointers**

Use . For struct **values**

Recall: struct pass by value

```
void changeName(struct studentT s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s.name, newName);  
}
```

```
void changeGpa(struct studentT s, float newGpa) {  
    s.gpa = newGpa;  
}
```

What is the output of this program?
Draw the stack diagram.

```
int main() {  
    struct studentT student1;  
  
    strcpy(student1.name, "Ruth");  
    student1.gpa = 3.5;  
  
    print("ORIGINAL", student1);  
    changeName(student1, "Anne");  
    print("AFTER CHANGE NAME", student1);  
  
    changeGpa(student1, 3.7);  
    print("AFTER CHANGE GPA", student1);  
  
    return 0;  
}
```

```
struct studentT {  
    char name[8];  
    float gpa;  
};
```

struct pass by pointer

```
void changeName(struct studentT* s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s->name, newName);  
    // DRAW STACK HERE  
}  
  
void changeGpa(struct studentT* s, float newGpa) {  
    s->gpa = newGpa;  
}
```

What syntax is different from the previous program?
What is the output of this program?

```
int main() {  
    struct studentT student1;  
  
    strcpy(student1.name, "Ruth");  
    student1.gpa = 3.5;  
  
    print("ORIGINAL", student1);  
    changeName(&student1, "Anne");  
    print("AFTER CHANGE NAME", student1);  
  
    changeGpa(&student1, 3.7);  
    print("AFTER CHANGE GPA", student1);  
  
    return 0;  
}
```

```

struct record {
    char tag;
    int values[3];
    float* score;
    struct record* next;
};

```

Assume 64 bit machine.

Variable / Expression	Size and rationale
<code>float x = 3.14;</code>	
<code>&x</code>	
<code>char* name = "CS";</code>	
<code>struct record r;</code>	
<code>r->tag</code>	
<code>r->values[1]</code>	
<code>r->values</code>	
<code>r->score</code>	
<code>*(r->score)</code>	
<code>r->next</code>	

```

struct record {
    char tag;
    int values[3];
    float* score;
    struct record* next;
};

```

Assume 32 bit machine.

Variable / Expression	Size and rationale
<code>float x = 3.14;</code>	4 bytes
<code>&x</code>	4 bytes
<code>char* name = "CS";</code>	char * points to a 1 byte char
<code>struct record first_record;</code>	1 char (1 byte) (3 ints x 4 bytes) 1 float ptr (4 bytes) 1 struct ptr (4 bytes) 21 bytes
<code>r->tag</code>	1 byte
<code>r->values[1]</code>	4 bytes
<code>r->values</code>	12 bytes
<code>r->score</code>	4 bytes
<code>*(r->score)</code>	4 bytes
<code>r->next</code>	21 bytes

Compiles or not?

```
void changeName(struct studentT s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s->name, newName);  
}
```

Compiles or not?

```
void changeName(struct studentT* s, char* newName) {  
    if (newName == NULL) {  
        return;  
    }  
    strcpy(s->name, newName);  
}
```

Compiles or not?

```
void changeName(struct student* s, float value) {  
    s->gpa = value;  
}
```

```
void changeName(struct student* s, float value) {  
    s.gpa = value;  
}
```

```
void changeName(struct student* s, float value) {  
    s->gpa = &value;  
}
```

Example: Checking for NULL

```
int main() {  
    int* value = NULL;  
    if (value != NULL) {  
        printf("value is %d\n", *value);  
    }  
  
    int a = 4;  
    value = &a;  
  
    if (value != NULL) {  
        printf("value is %d\n", *value);  
    }  
}
```

Return values

When we return a value, the *value* is copied

When we return a pointer, the *address* is copied

If the address refers to memory that is destroyed when the function stack pops, the address will be invalid

Return value gotchas: Safe or not?

```
char* code(int v) {  
    char msg[16];  
    if (v == 0) strcpy(msg, "val0");  
    else if (v == 1) strcpy(msg, "val1");  
    else if (v == 2) strcpy(msg, "val2");  
    return msg;  
}  
  
int main() {  
    int val = 1;  
    char* printme = code(val);  
    printf("%s\n", printme);  
    return 0;  
}
```

Draw the function stack/heap.
Is this return value safe or not?

Return value gotchas: Safe or not?

```
char* code(int v) {  
    if (v == 0) return "val0";  
    else if (v == 1) return "val1";  
    return "val2";  
}  
  
int main() {  
    int val = 1;  
    char* printme = code(val);  
    printf("%s\n", printme);  
    return 0;  
}
```

Draw the function stack/heap. Assume val = 1
Is this return value safe or not?

Return value gotchas: Safe or not?

```
char* code(int v) {  
    char* msg = malloc(sizeof(char) * 16);  
    if (v == 0) strcpy(msg, "val0");  
    else if (v == 1) strcpy(msg, "val1");  
    else if (v == 2) strcpy(msg, "val2");  
    return msg;  
}  
  
int main() {  
    int val = 1;  
    char* str = code(val);  
    printf("%s\n", str);  
    return 0;  
}
```

Draw the function stack/heap.
Is this return value safe or not?
What about memory leaks?

Return value gotchas: Safe or not?

```
void code(int v, char* msg) {  
    if (v == 0) strcpy(msg, "val0");  
    else if (v == 1) strcpy(msg, "val1");  
    else if (v == 2) strcpy(msg, "val2");  
}
```

```
int main() {  
    int val = 1;  
  
    char msg[16];  
    code(val, msg);  
    printf("%s\n", msg);  
    return 0;  
}
```

Standard I/O

Three default streams

- `stdin` (`scanf`, `fscanf`)
- `stdout` (`printf`, `fprintf`)
- `stderr` (`fprintf(stderr, <format string>, <data>))`)

`stdin`, **`stdout`**, **`stderr`** are tied to files with descriptors 0, 1, and 2 respectively

Standard I/O

At the command line we can redirect these streams to files

- `./a.out < infile.txt`
- `./a.out > outfile.txt`
- `./a.out &> allfile.txt` (all output)
- `./a.out 2> errfile.txt` (error output only)
- `./a.out 1> outfile.txt` is the same as `./a.out > outfile.txt`

Standard I/O (C)

```
int a;  
char word[32];  
printf("Enter an integer: ");  
scanf(" %d", &a);  
  
printf("Enter a string: ");  
scanf(" %s", word);  
  
printf("\nYou entered: %d %s\n",  
      a, word);  
fprintf(stderr, "Test printing an error\n");
```

```
int a;  
char word[32];  
fprintf(stdout, "Enter an integer: ");  
fscanf(stdin, " %d", &a);  
  
fprintf(stdout, "Enter a string: ");  
fscanf(stdin, " %s", word);  
  
fprintf(stdout, "\nYou entered: %d %s\n",  
      a, word);  
fprintf(stderr, "Test printing an error\n");
```

printf(...) is the same as fprintf(stdout, ...)

When we redirect output/input, they are fed into/out of these commands
(demo)

File I/O (Text Files)

Reading/writing files is done using a **file pointer**

```
#include <stdio.h>

FILE *infile; FILE *outfile;

infile = fopen("input.txt", "r");
if (infile == NULL) {
    printf("Error: unable to open file %s\n", "input.txt");
    exit(1);
}

outfile = fopen("output.txt", "w");
if (outfile == NULL) {
    printf("Error: unable to open outfile\n");
    exit(1);
}

fclose(infile);
fclose(outfile);
```

fopen initializes the file pointer to the beginning of the file.

Calling fclose is important to avoid a memory leak

Reading Text Files (C)

```
const char* filename = "grades.txt";
FILE* fp = fopen(filename, "r");

if (!fp) // fp is NULL on error
{
    printf("Cannot load file: %s\n", filename);
    return 1;
}

int num = 0;
float sum = 0;
char line[1024];

while (fgets(line, 1024, fp))
{
    int grade;
    sscanf(line, " %d", &grade);
    sum += grade;
    num++;
}

printf("The average is %.2f\n", sum/num);
fclose(fp);
```

```
99
79
51
92
56
89
63
```

Helpful text file commands

- `fprintf`, `fscanf`
- `fgets`, `fputs` (gets/puts a line of input)
- `fgetc`, `fputc` (gets/puts a single character)

Helpful string commands

- `strlen`: get the length
- `strcpy`: copy one string to another
- `strcmp`: returns 0 if two strings are the same
- `strstr`: check if a string contains a substring
- `strtok`: tokenize a string

How does redirected input/output compare to reading/writing from files?

Explicit file I/O: **the program opens the file.**

Redirection: **the shell connects a file to the program's existing standard streams.**